

WinForms / Avalonia “Notepad” Application / Fibonacci BigInt Text Reader
Cpt S 321 Homework Assignment
Washington State University

Read the entire HW before starting to work on it.

Overview of the steps to follow:

1. Create a branch called “Branch_HW3” and work in this branch for this assignment.
2. Read the Homework **Grading Rubric and the Points Breakdown** that appear at the end of this document.
3. **Work on the Assignment Instructions below.**
4. **Create a pdf document with your AI statement for the HW (call it “AI_HW3”), stating whether or not you used GenAI and how and upload it in the “AI_Statements” folder. If you used GenAI for this HW, 1) detail the process you followed in this document and 2) follow the guidelines for committing code generated by AI and adapted by you as outlined in the syllabus in the AI Statement section.**
5. Once you are done and before the deadline, tag the version that you would want us to grade with the assignment number. For example, “HW1”, “HW2”, etc.
6. **Record a video to demo your HW (call it “Demo_HW3-FirstName_LastName”).**
7. On Canvas -> Assignments -> Assignments -> **Upload your pre-recorded video by the HW deadline.**
8. Sign up for a live demo with the TAs before the deadline.

Assignment Instructions:

Read all the instructions *carefully* before you write any code.

Using Visual Studio (or Rider if you are doing a cross-platform development), create a WinForms (Avalonia MVVM) application that can load and save plain text files. This will be much like a simplified version of the Notepad utility application that comes with Microsoft Windows. This app will also include the ability to load text from a non-file source (Fibonacci in-memory sequence generator).

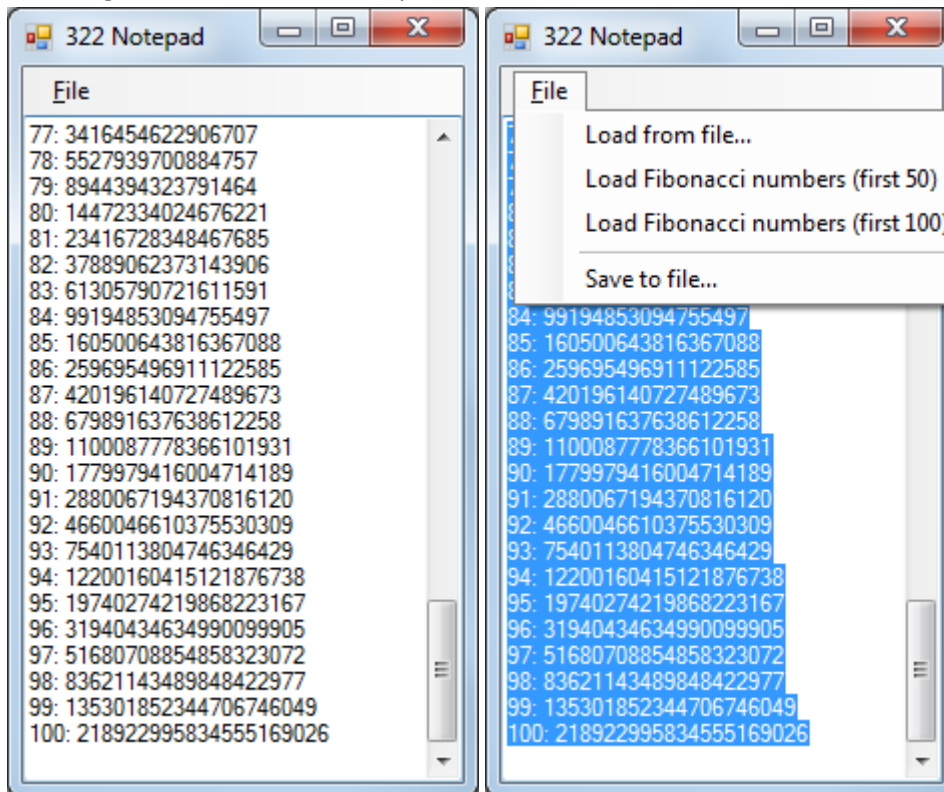
Interface:

You are free to design the interface the way you wish for the most part, but you must include the following:

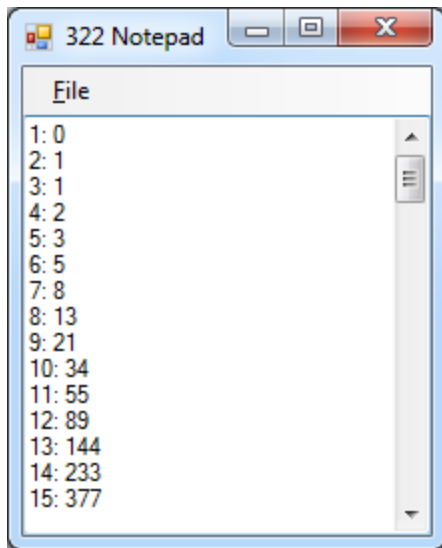
- a prominent text area that contains the text
 - Use a TextBox
 - Set the Multiline property to true for WinForms (multiline is the default for Avalonia)
 - For WinForms, set the ScrollBars property to Vertical or Both so that vertical scrolling can occur

- buttons or menu options to load and save files
 - You will need 3 options for loading text, one from file, two from another source. This is discussed more below.
 - For Avalonia, check the documentation on Menus [here](#); we will use a static menu.
 - selecting files to open/save
 - for WinForms, use [OpenFileDialog/SaveFileDialog](#) for selecting files to open/save. Can be created in code or at design time by dragging them in from the toolbox
 - for Avalonia, we need to manually bind the code to the user interface in the .axaml file.
- As we go, we will learn more about good practices for interactions between the user interface and the rest of the code but for now, we will follow a template provided in addition to the instructions without worrying too much about why things are defined this way. Check the documentation for File Dialogs in Avalonia [here](#).

The image below shows an example of what the interface could look like:



Here's another image of the application scrolled up to the beginning numbers:



Saving Functionality:

Your application needs to provide the ability to save the text to a text file. Provide a button or menu option that brings up a `SaveFileDialog`, prompting the user for a file name. If the user clicks OK in that dialog, which you'll determine by checking the return value of the `ShowDialog` method for WinForms, then save the text in the text box to that file. You can save it using streams, text-writers, or any other method you see fit (it's the loading that's going to be more specific). For Avalonia, check the example [here](#).

Loading Functionality:

1. Generic Loading Function

Write a function called "LoadText" in your main form's .CS file for WinForm or in your `MainWindowViewModel`'s .CS file for Avalonia. This will take a `System.IO.TextReader` object as a parameter. It should read all the text from the `TextReader` object and put it in the text box in your interface.

- a. Declare and implement the method mentioned above:

```
private void LoadText(TextReader sr)
```

- b. Use either [ReadToEnd](#) or [ReadLine](#) (in a loop) to load all text from the reader and put it in the text box. This should replace any content already in the text box.

2. Loading from file

Add the option to load text from a file. You can use a stream reader to open the file and pass it to your loading function. Remember that [StreamReader](#) inherits from `TextReader` and has a [constructor](#) that takes a file name as a parameter.

3. Loading from FibonacciTextReader

Write a class named `FibonacciTextReader` that inherits from the `System.IO.TextReader`. Since you've implemented a loading function in part 1 that takes a text reader, you'll be able to pass this object to your loading function after creating an instance of it. This class must:

- a. Inherit from the `TextReader` class

- b. Make it have a constructor that takes an integer as a parameter indicating the maximum number of lines available (the maximum number of numbers in the sequence that you can generate).
 - c. Override the ReadLine method which delivers the next number (as a string) in the Fibonacci sequence. You need logic in this function to handle the first two numbers as special cases. You must return null after the nth call, where n is the integer value passed to the constructor.
 - d. Override the ReadToEnd method if you used it back in the implementation for part 1. Implement it such that it repeatedly calls ReadLine and concatenates all the lines together. You can use the [System.Text.StringBuilder](#) class to append the lines together into one string.
 - e. Use the [System.Numerics.BigInteger](#) struct for the sequence numbers. The numbers get beyond what a 64-bit integer can store very quickly, so you need this to accurately compute the first 100 numbers in the sequence.
 - f. Optional: put the line number before the sequence number as shown in the sample app screenshot.
4. Add three menu options (or buttons if you want) in your interface to allow for demonstration of the above functionality.
 - a. There needs to be an option to load from a file. This should prompt the user for a file name (use OpenFileDialog for convenience) and then load it as described in #2 above.
 - b. There needs to be an option to load the first 50 numbers of the Fibonacci sequence
 - c. There needs to be an option to load the first 100 numbers of the Fibonacci sequence
 - d. (note: parts b and c should have almost identical code, just with a different number passed to the FibonacciTextReader constructor)

Homework Grading Rubric and Points Breakdown	
CR1: Functionality (4 points)	● Fulfill all the requirements above with no inaccuracies in the output and no crashes. ○ 1.5 points for correct saving functionality ○ 2.5 points for correct loading function
CR2 Quality of Version Control (1 point)	● Homework should be built iteratively (i.e., one feature at a time, not in one huge commit). ● Each commit should have cohesive functionality. ● Commit messages should concisely describe what is being committed. ● TDD should be used (i.e, write and commit tests first and then implement and commit functionality). ● Include "TDD" in all commit messages with tests that are written before the functionality is implemented.

	• Use of a .gitignore. • Commenting is done as the homework is built (i.e, there is commenting added in each commit, not done all at once at the end).
CR3: Quality of Identifiers (1 point)	• No underscores in names of classes, attributes, and properties. • No numbers in names of classes or tests. • Identifiers should be descriptive. • Project names should make sense. • Class names and method names use PascalCasing. • Method arguments and local variables use camelCasing. • No Linguistic Antipatterns or Lexicon Bad Smells.
CR4: Existence and Quality of Comments (1 point)	• Every method, attribute, type, and test case has a leading comment block with a minimum of <summary>, <returns>, <param>, and <exception> filled in as applicable. • All comment blocks use the format that is generated when typing “///” on the line above each entity. • There is useful inline commenting <u>in addition to leading comment blocks</u> that explains how the algorithm is implemented as needed.
CR5: Quality of Code (1 point)	• Each file should only contain one public class. • Correct use of access modifiers. • Classes are cohesive. • Namespaces make sense. • Code is easy to follow. • StyleCop is installed and configured correctly for all projects in the solution and all warnings are resolved. If any warnings are suppressed, a good reason must be provided. • Use of appropriate design patterns and software principles seen in class.
CR6: Existence and Quality of Tests (1 point)	• Normal, boundary, and overflow/error cases should be tested – tests should be written with NUnit. • Test should be modularized (i.e, have a separate test case for each functionality - do not combine them into one large test case). • <i>Note: In assignments with a GUI, we do not require testing of the GUI itself.</i>
CR7: Demo video – 5 min max (1 point)	• Check the “Pre-recorded demos instructions” in the additional resources for details on the items below. • Version control history overview (< 30 sec.) • AI statement (< 30 sec.)

	• Design of the solution (2 min max) • Features showcase (2 min max)
--	--